



Patrón de Diseño Observer

Observador · Publicación-Suscripción · Modelo-patrón · Event-Subscriber · Listener

Agenda de la Presentación

01

Propósito

Qué es el patrón Observer y para qué sirve

03

Solución

Mecanismo de suscripción y notificación

05

Estructura

Componentes del patrón y sus roles

07

Aplicabilidad

Cuándo y cómo usar el patrón

02

Problema

El conflicto entre cliente y tienda

04

Analogía

Suscripciones a revistas y periódicos

06

Pseudocódigo

Implementación completa con ejemplo

08

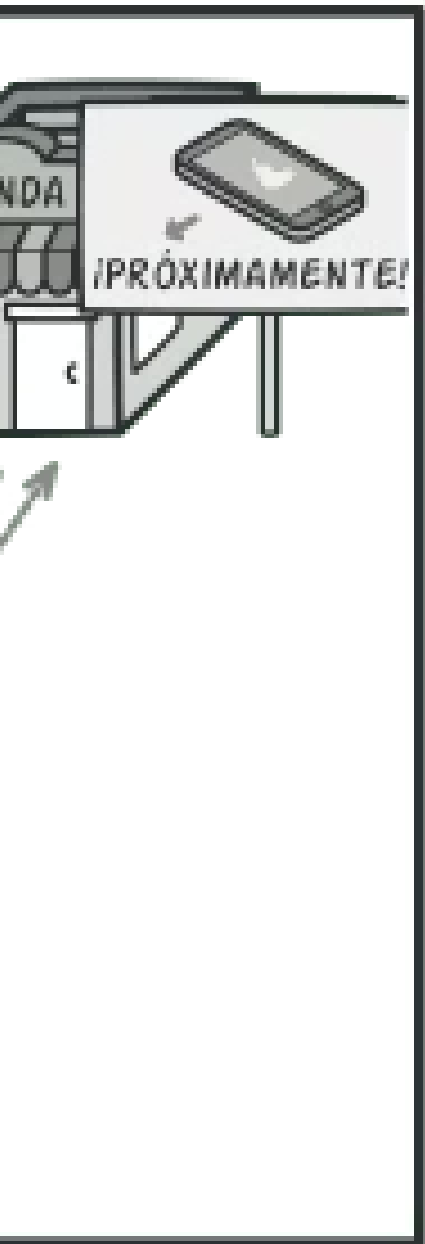
Pros, contras y relaciones

Ventajas, desventajas y patrones relacionados

El Problema: Cliente vs. Tienda

Imagina que tienes dos tipos de objetos: un objeto **Cliente** y un objeto **Tienda**. El cliente está muy interesado en una marca particular de producto (digamos, un nuevo modelo de iPhone) que estará disponible en la tienda muy pronto.

El cliente puede visitar la tienda cada día para comprobar la disponibilidad del producto. Pero, mientras el producto está en camino, **la mayoría de estos viajes serán en vano.**



Visita a la Tienda vs. Envío de Spam

1 Opción A: El cliente visita

El cliente pierde tiempo comprobando la disponibilidad del producto cada día, realizando viajes innecesarios a la tienda.

2 Opción B: La tienda notifica a todos

La tienda envía cientos de correos (spam) a todos los clientes cada vez que hay un producto nuevo, molestando a quienes no están interesados.

El Conflicto Central

Parece que nos encontramos ante un conflicto. O el cliente pierde tiempo comprobando la disponibilidad del producto, o bien la tienda desperdicia recursos notificando a los clientes equivocados.

Necesitamos un mecanismo que permita notificar **solo a los interesados**, sin desperdiciar recursos ni tiempo.



La Solución: Sujeto y Suscriptores

Sujeto / Notificador

El objeto que tiene un estado interesante y va a notificar a otros objetos los cambios en su estado. También llamado **publicador**.

Suscriptores

El resto de los objetos que quieren conocer los cambios en el estado del notificador. Se suscriben para recibir eventos.

El patrón Observer sugiere que añadas un **mecanismo de suscripción** a la clase notificadora para que los objetos individuales puedan suscribirse o cancelar su suscripción a un flujo de eventos.

intame,
or!

- subscri

...

+ addSub

+ remove

mbién!

Mecanismo de Suscripción

Este mecanismo consiste en dos elementos fundamentales:

1

Campo Matriz

Un campo para almacenar una **lista de referencias** a objetos suscriptores.

2

Métodos Públicos

Varios métodos públicos que permiten **añadir suscriptores** y **eliminarlos** de esa lista.

Un mecanismo de suscripción permite a los objetos individuales suscribirse a notificaciones de eventos.

Notificación a Suscriptores

Cuando le sucede un **evento importante** al notificador, recorre sus suscriptores y llama al método de notificación específico de sus objetos.

Las aplicaciones reales pueden tener **decenas de clases suscriptoras diferentes** interesadas en seguir los eventos de la misma clase notificadora.

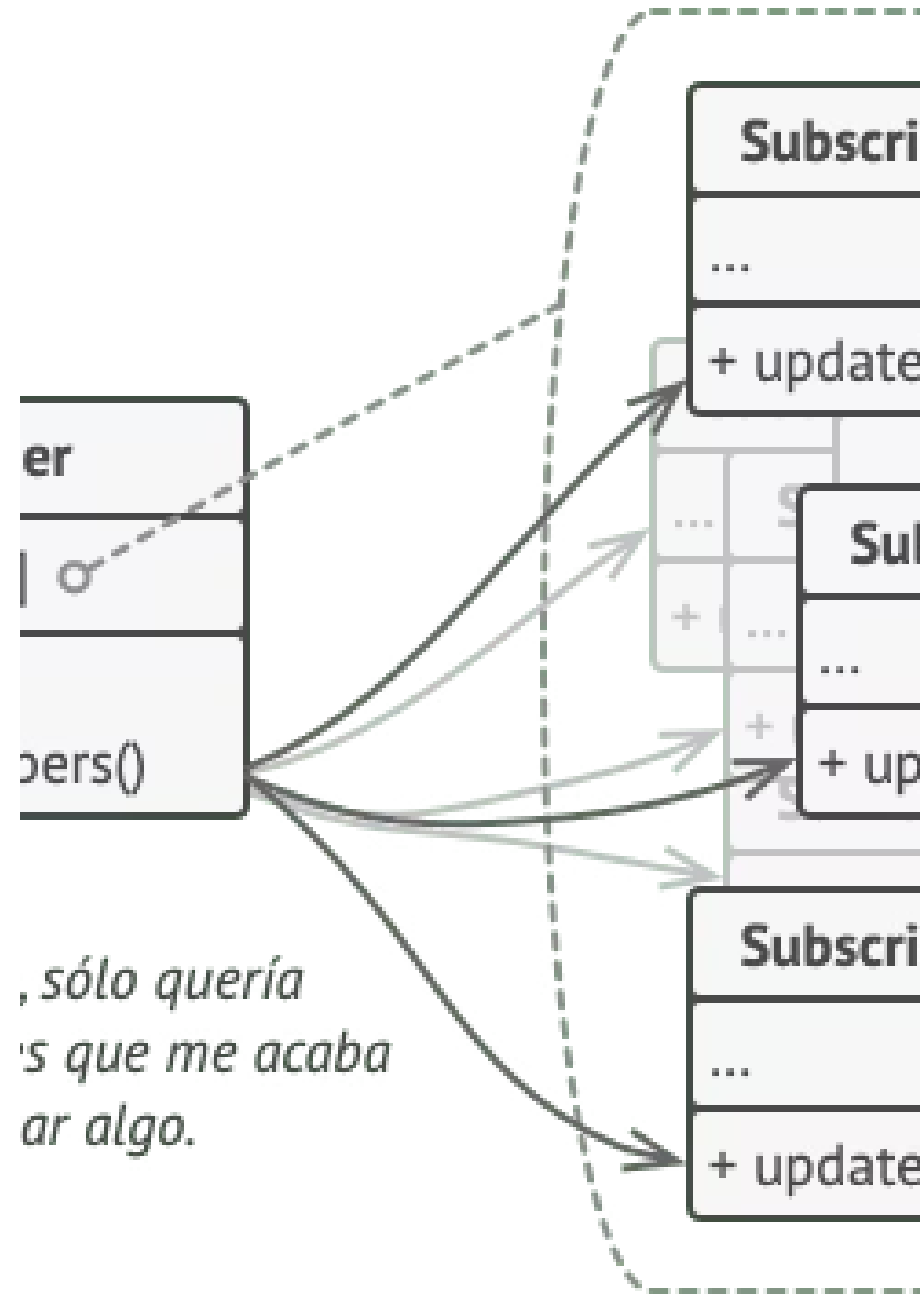
No querrás acoplar la notificadora a todas esas clases.

- ❏ Es fundamental que todos los suscriptores implementen la **misma interfaz** y que el notificador únicamente se comunique con ellos a través de esa interfaz. Esta interfaz debe declarar el método de notificación junto con parámetros para pasar información contextual.

Diagrama de Notificación

El notificador notifica a los suscriptores invocando el método de notificación específico de sus objetos.

Si tu aplicación tiene varios tipos diferentes de notificadores y quieres hacer a tus suscriptores compatibles con todos ellos, puedes hacer que **todos los notificadores sigan la misma interfaz**. Esta interfaz sólo tendrá que describir algunos métodos de suscripción, permitiendo a los suscriptores observar los estados sin acoplarse a clases concretas.



CAPÍTULO 4 ANALOGÍA

Analogía en el Mundo Real

Suscripciones a revistas y periódicos.



Cómo Funciona la Suscripción

1 Te suscribes

Si te suscribes a un periódico o una revista, ya no necesitarás ir a la tienda a comprobar si el siguiente número está disponible.

3 Lista de suscriptores

El notificador mantiene una lista de suscriptores y sabe qué revistas les interesan.

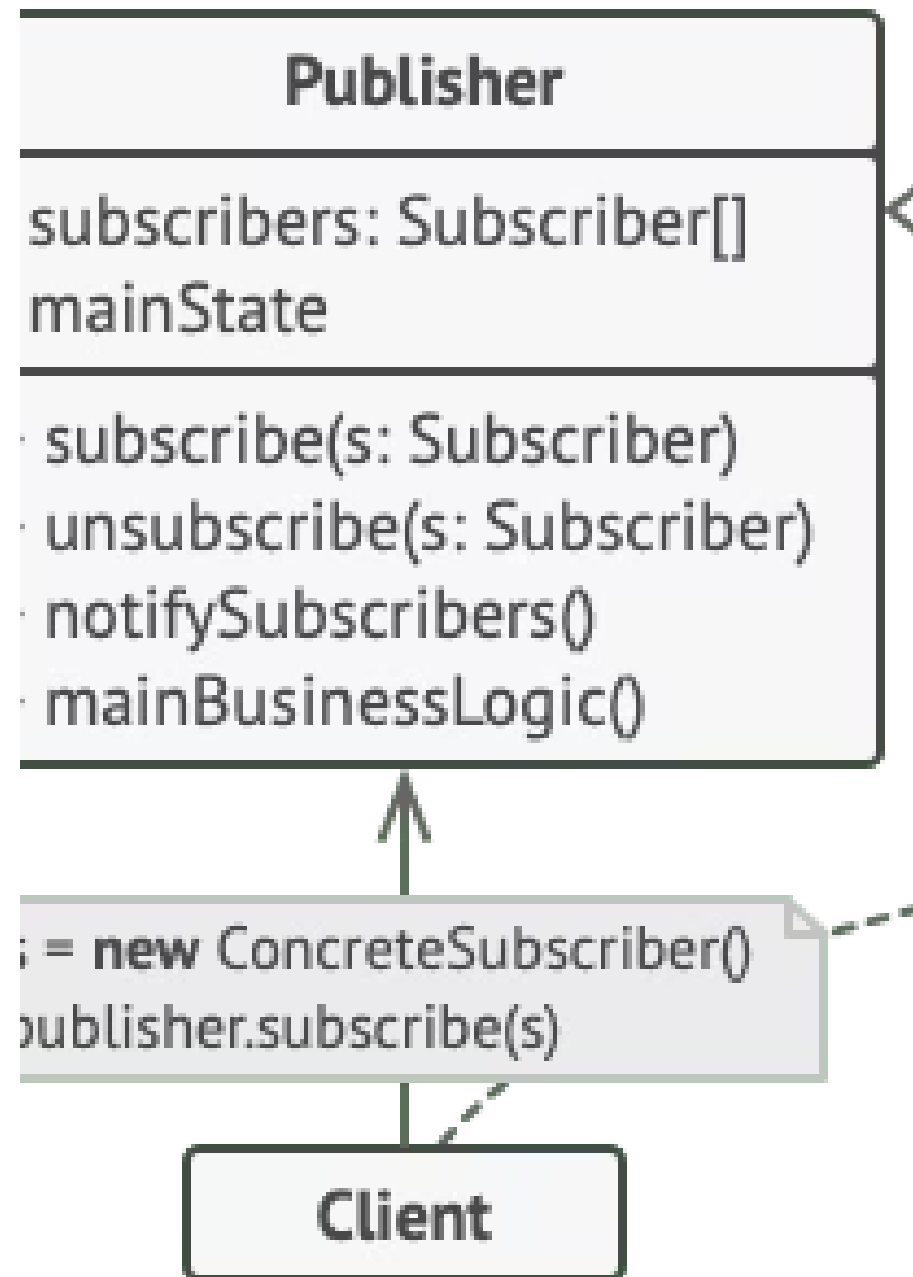
2 Recibes notificaciones

El notificador envía nuevos números directamente a tu buzón justo después de la publicación, o incluso antes.

4 Cancelación libre

Los suscriptores pueden abandonar la lista en cualquier momento si quieren que el notificador deje de enviarles nuevos números.

Estructura del Patrón Observer



Componentes de la Estructura

1

Notificador

Envía eventos de interés a otros objetos. Contiene una infraestructura de suscripción que permite a nuevos y antiguos suscriptores abandonar la lista.

2

Recorrido de la lista

Cuando sucede un nuevo evento, el notificador recorre la lista de suscripción e invoca el método de notificación declarado en la interfaz suscriptora en cada objeto.

3

Interfaz Suscriptora

Declara la interfaz de notificación. En la mayoría de los casos, consiste en un único método **actualizar** con varios parámetros para detalles del evento.

Componentes de la Estructura (cont.)

1

Suscriptores Concretos

Realizan acciones en respuesta a las notificaciones emitidas por el notificador. Todas estas clases deben implementar la misma interfaz para que el notificador no esté acoplado a clases concretas.

2

Información Contextual

Los suscriptores necesitan cierta información contextual para manejar la actualización. Los notificadores pasan información de contexto como argumentos del método de notificación, o pueden pasarse a sí mismos como argumento.

3

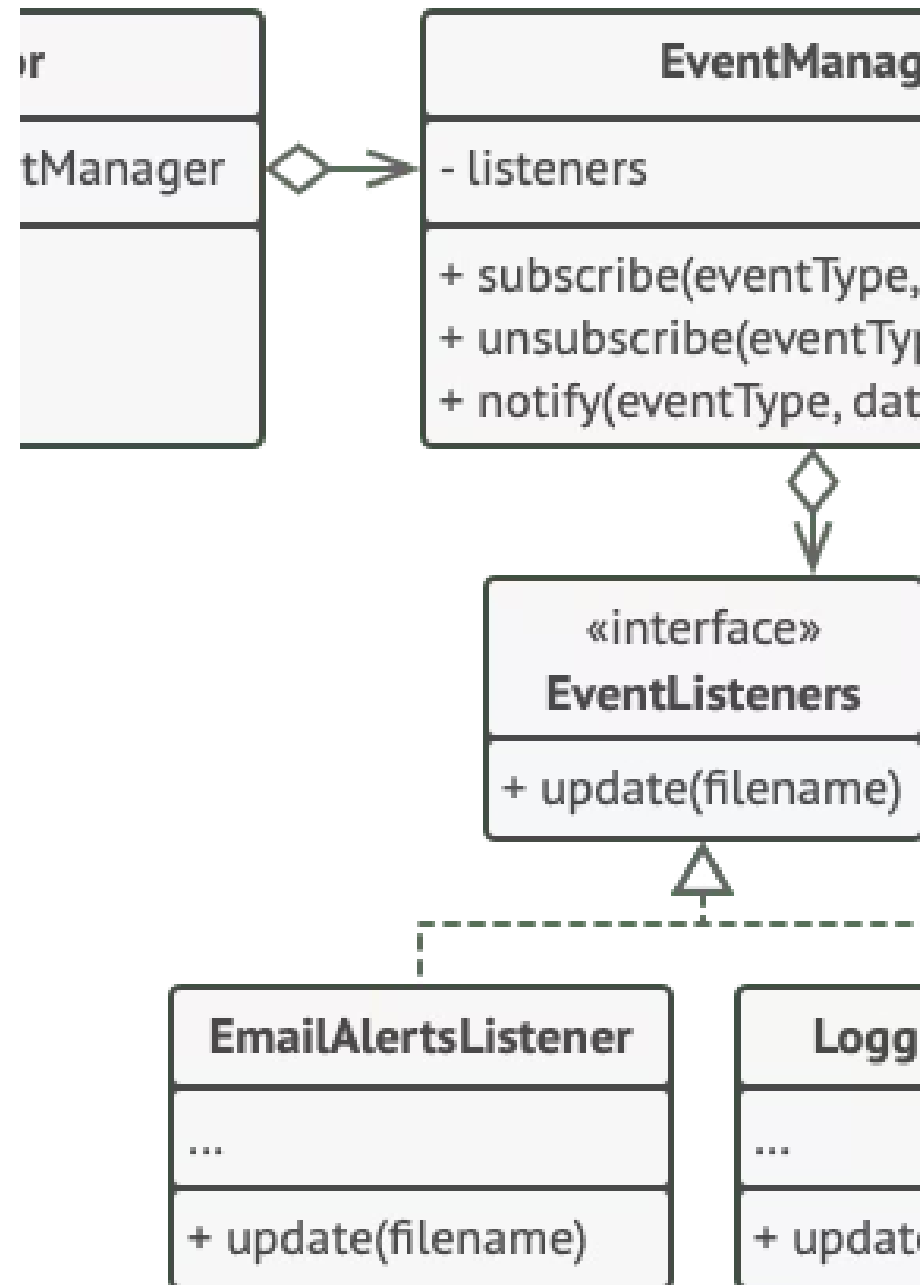
Cliente

Crea objetos tipo notificador y suscriptor por separado y después registra a los suscriptores para las actualizaciones del notificador.

Pseudocódigo: Editor de Texto

Notificar a objetos sobre eventos que suceden a otros objetos.

La lista de suscriptores se compila dinámicamente: los objetos pueden empezar o parar de escuchar notificaciones durante el tiempo de ejecución. La clase editora **delega** el trabajo de gestión de suscripciones a un objeto ayudante especial (EventManager).



Clase EventManager (Notificadora Base)

La clase notificadora base incluye código de gestión de suscripciones y métodos de notificación.

```
// La clase notificadora base incluye código de gestión de
// suscripciones y métodos de notificación.
class EventManager is
  private field listeners: hash map of event types and listeners

  method subscribe(eventType, listener) is
    listeners.add(eventType, listener)

  method unsubscribe(eventType, listener) is
    listeners.remove(eventType, listener)

  method notify(eventType, data) is
    foreach (listener in listeners.of(eventType)) do
      listener.update(data)
```

Clase Editor (Notificador Concreto)

El notificador concreto contiene lógica de negocio real. Utiliza composición para integrar la lógica de suscripción.

```
// El notificador concreto contiene lógica de negocio real, de
// interés para algunos suscriptores. Podemos derivar esta clase
// de la notificadora base, pero esto no siempre es posible en
// el mundo real porque puede que la notificadora concreta sea
// ya una subclase. En este caso, puedes modificar la lógica de
// la suscripción con composición, como hicimos aquí.
```

```
class Editor is
```

```
    public field events: EventManager
```

```
    private field file: File
```

```
    constructor Editor() is
```

```
        events = new EventManager()
```

```
    // Los métodos de la lógica de negocio pueden notificar los
```

```
    // cambios a los suscriptores.
```

```
    method openFile(path) is
```

```
        this.file = new File(path)
```

```
        events.notify("open", file.name)
```

```
    method saveFile() is
```

```
        file.write()
```

```
        events.notify("save", file.name)
```

```
    // ...
```

Interfaz EventListener

Si tu lenguaje de programación soporta tipos funcionales, puedes sustituir toda la jerarquía suscriptora por un grupo de funciones.

```
// Aquí está la interfaz suscriptora. Si tu lenguaje de
// programación soporta tipos funcionales, puedes sustituir toda
// la jerarquía suscriptora por un grupo de funciones.
interface EventListener is
  method update(filename)
```

Suscriptores Concretos

Los suscriptores concretos reaccionan a las actualizaciones emitidas por el notificador al que están unidos.

```
// Los suscriptores concretos reaccionan a las actualizaciones
```

```
// emitidas por el notificador al que están unidos.
```

```
class LoggingListener implements EventListener is
```

```
    private field log: File
```

```
    private field message: string
```

```
    constructor LoggingListener(log_filename, message) is
```

```
        this.log = new File(log_filename)
```

```
        this.message = message
```

```
    method update(filename) is
```

```
        log.write(replace('%s',filename,message))
```

```
class EmailAlertsListener implements EventListener is
```

```
    private field email: string
```

```
    private field message: string
```

```
    constructor EmailAlertsListener(email, message) is
```

```
        this.email = email
```

```
        this.message = message
```

```
    method update(filename) is
```

```
        system.email(email, replace('%s',filename,message))
```

Clase Application (Cliente)

Una aplicación puede configurar notificadores y suscriptores durante el tiempo de ejecución.

```
// Una aplicación puede configurar notificadores y suscriptores
// durante el tiempo de ejecución.
class Application is
    method config() is
        editor = new Editor()

        logger = new LoggingListener(
            "/path/to/log.txt",
            "Someone has opened the file: %s")
        editor.events.subscribe("open", logger)

        emailAlerts = new EmailAlertsListener(
            "admin@example.com",
            "Someone has changed the file: %s")
        editor.events.subscribe("save", emailAlerts)
```

- ❏ Añadir nuevos suscriptores al programa **no requiere cambios** en clases notificadoras existentes, siempre y cuando trabajen con todos los suscriptores a través de la misma interfaz.

¿Cuándo Utilizar el Patrón Observer?

Cambios de estado desconocidos

Utiliza el patrón cuando los cambios en el estado de un objeto puedan necesitar cambiar otros objetos y el grupo de objetos sea **desconocido de antemano** o cambie dinámicamente. Ejemplo: clases de interfaz gráfica, botones personalizados que activan código cliente al ser pulsados.

Observación temporal

Utiliza el patrón cuando algunos objetos de tu aplicación deban observar a otros, pero sólo durante un **tiempo limitado** o en casos específicos. La lista de suscripción es dinámica: los suscriptores pueden unirse o abandonar la lista cuando lo deseen.

Cómo Implementar el Patrón Observer

Paso 1: Dividir la lógica

Repasa tu lógica de negocio y divídela en dos partes: la funcionalidad central (notificador) y el resto (clases suscriptoras).

1

2

Paso 2: Interfaz suscriptora

Declara la interfaz suscriptora con al menos un único método **actualizar**.

Paso 3: Interfaz notificadora

Declara la interfaz notificadora con métodos para añadir y eliminar suscriptores. Los notificadores deben trabajar con suscriptores únicamente a través de la interfaz.

3

4

Paso 4: Lista de suscripción

Coloca la lista y los métodos de suscripción en una clase abstracta o en un objeto separado (composición) si aplicas el patrón a una jerarquía existente.

Paso 5: Notificadores concretos

Crea clases notificadoras concretas que notifiquen a todos sus suscriptores cuando suceda algo importante.

5

6

Paso 6: Métodos de actualización

Implementa los métodos de notificación en suscriptores concretos. La información de contexto puede pasarse como argumento o el notificador puede pasarse a sí mismo.

Paso 7: Configuración del cliente

El cliente debe crear todos los suscriptores necesarios y registrarlos con los notificadores adecuados.

7

Pros y Contras

✓ Ventajas

- **Principio de abierto/cerrado.** Puedes introducir nuevas clases suscriptoras sin tener que cambiar el código de la notificadora (y viceversa si hay una interfaz notificadora).
- Puedes establecer **relaciones entre objetos** durante el tiempo de ejecución.

✗ Desventajas

- Los suscriptores son notificados en un **orden aleatorio**.

Relaciones con Otros Patrones

[Chain of Responsibility](#), [Command](#), [Mediator](#) y [Observer](#) abordan distintas formas de conectar emisores y receptores de solicitudes:

Chain of Responsibility

Pasa una solicitud secuencialmente a lo largo de una cadena dinámica de receptores hasta que uno la gestiona.

Command

Establece conexiones unidireccionales entre emisores y receptores.

Mediator

Elimina conexiones directas, forzando comunicación indirecta a través de un objeto mediador.

Observer

Permite a los receptores suscribirse o darse de baja dinámicamente a la recepción de solicitudes.

Mediator vs. Observer

La meta del **Mediator** es eliminar dependencias mutuas entre componentes, haciéndolos dependientes de un único objeto mediador. La meta del **Observer** es establecer conexiones dinámicas de un único sentido donde algunos objetos actúan como subordinados de otros.

Hay una implementación popular del Mediator basada en Observer: el objeto mediador juega el papel de notificador y los componentes actúan como suscriptores. Pero también puedes vincular permanentemente todos los componentes al mismo mediador, lo cual no se parece al Observer pero sigue siendo Mediator.